

# Lab 8 Study Notes

## Understanding the AlexNet Ablation Assignment

Siddharth Narayan Mishra (123CS0189)

September 30, 2026

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>The big picture</b>                             | <b>3</b> |
| 1.1      | What the assignment is really asking . . . . .     | 3        |
| 1.2      | Why it is called an ablation study . . . . .       | 3        |
| 1.3      | The four things you measure . . . . .              | 3        |
| <b>2</b> | <b>Background you need</b>                         | <b>3</b> |
| 2.1      | The dataset and the three splits . . . . .         | 3        |
| 2.2      | Why so much is fixed . . . . .                     | 4        |
| 2.3      | Preprocessing . . . . .                            | 4        |
| 2.4      | Training vocabulary . . . . .                      | 5        |
| 2.5      | Cross-entropy loss . . . . .                       | 5        |
| 2.6      | Adam . . . . .                                     | 6        |
| 2.7      | Dropout . . . . .                                  | 6        |
| <b>3</b> | <b>The architecture</b>                            | <b>6</b> |
| 3.1      | Reading the layers . . . . .                       | 6        |
| 3.2      | Why the sizes are 64, 32, 16, 8 . . . . .          | 7        |
| 3.3      | Flatten and fully connected layers . . . . .       | 7        |
| 3.4      | Is the baseline the real AlexNet? . . . . .        | 7        |
| 3.5      | What ReLU does . . . . .                           | 7        |
| <b>4</b> | <b>Parameters versus computation</b>               | <b>7</b> |
| 4.1      | Counting parameters . . . . .                      | 8        |
| 4.2      | Counting computation . . . . .                     | 8        |
| 4.3      | Backward pass . . . . .                            | 9        |
| <b>5</b> | <b>Generalisation, overfitting and convergence</b> | <b>9</b> |
| 5.1      | Definitions . . . . .                              | 9        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 5.2      | Why overfitting is likely in this setup . . . . .                     | 10        |
| 5.3      | How to read the curves . . . . .                                      | 10        |
| <b>6</b> | <b>Expected results and the reasoning behind them</b>                 | <b>10</b> |
| 6.1      | Experiment 1: baseline . . . . .                                      | 10        |
| 6.2      | Experiment 2: smaller classifier . . . . .                            | 10        |
| 6.3      | Experiment 3: narrower convolutions . . . . .                         | 11        |
| 6.4      | The comparison you are ultimately asked to make . . . . .             | 11        |
| 6.5      | What actually happened (local CPU run) . . . . .                      | 11        |
| <b>7</b> | <b>Reading the three plots</b>                                        | <b>12</b> |
| <b>8</b> | <b>The proposed modification</b>                                      | <b>12</b> |
| 8.1      | A candidate: add batch normalisation after each convolution . . . . . | 12        |
| 8.2      | Alternatives, if you want a second angle . . . . .                    | 13        |
| <b>9</b> | <b>Likely questions and short answers</b>                             | <b>13</b> |

# 1 The big picture

## 1.1 What the assignment is really asking

You train three small convolutional networks on the same small slice of CIFAR-10 and compare them. The three networks differ in exactly one place each:

- **Baseline:** the reference network.
- **Experiment 2:** the same network, but the fully connected (FC) head at the end is made smaller.
- **Experiment 3:** the same network, but every convolutional layer has half as many filters.

The question behind the whole assignment is: *where does a CNN keep its capacity, and what does it cost to remove some of it?* Capacity means how complicated a function the network can represent. More capacity lets a model fit the training data better, but also makes it easier to memorise the training data instead of learning general patterns.

## 1.2 Why it is called an ablation study

An **ablation** removes or shrinks one component and measures what changes. It only works if *nothing else changes*. That is why the assignment is so strict about fixed seeds, the same data split, the same optimizer and the same number of epochs. If two runs differed in the split as well as the architecture, you could not tell which one caused a difference in accuracy.

## 1.3 The four things you measure

1. **Accuracy:** fraction of images classified correctly.
2. **Loss:** cross-entropy, a smoother measure of how confident and correct the predictions are.
3. **Model complexity:** the number of trainable parameters.
4. **Computational cost:** wall-clock training time (and, in theory, the number of arithmetic operations).

Notice that complexity and computational cost are *different things*. Section 4 shows this is the main lesson of the assignment.

# 2 Background you need

## 2.1 The dataset and the three splits

CIFAR-10 has 60 000 small colour images ( $32 \times 32$ ) in 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. The assignment uses a small subset:

| Split      | Size   | Drawn from                                       |
|------------|--------|--------------------------------------------------|
| Training   | 12 000 | original training set                            |
| Validation | 2 000  | original training set (no overlap with training) |
| Test       | 2 000  | original test set                                |

Each split has a distinct job:

- **Training set:** the network’s weights are updated using it.
- **Validation set:** used to *compare models and make decisions* (which is better, has it converged, is it overfitting). It is never trained on.
- **Test set:** touched once, at the very end, to report an honest final number. If you used it to pick a model, it would no longer be an unbiased estimate, because you would have effectively tuned on it. This is why the assignment repeats that the test set must not be used for selection.

Random chance on 10 balanced classes is 10% accuracy, and the untrained network’s loss should start near  $\ln 10 \approx 2.303$ . Both are useful sanity checks when you look at your first epoch.

## 2.2 Why so much is fixed

- **Seed 42 for the split:** all three models are trained and validated on exactly the same images.
- **Seed reset before building each model:** the initial random weights come from the same random stream, so no model gets a luckier starting point through the random number generator.
- **Fixed generator for shuffling:** every model sees the mini-batches in the same order.
- **No tuning per model:** if you tuned the learning rate for one model but not the others, a difference in results would partly reflect your tuning effort rather than the architecture.

An honest caveat to mention in the analysis: this is still *one run per model*. Different weights initialisation for a different seed would give slightly different numbers. With only 2 000 validation images, one image is worth 0.05 percentage points, and the random sampling error on an accuracy near 70% is about

$$\sqrt{\frac{0.7 \times 0.3}{2000}} \approx 1.0 \text{ percentage point.}$$

So differences smaller than about 1 to 2 points between models should *not* be described as real improvements.

## 2.3 Preprocessing

- **Resize to 64×64:** the original images are 32×32. Upscaling does not add information, but it gives the network bigger feature maps to work with. It also makes the arithmetic cost of the convolutions four times larger.
- **Normalisation:** each colour channel has the dataset mean subtracted and is divided by the standard deviation,  $x' = (x - \mu)/\sigma$ . This centres the inputs around zero with roughly unit spread. Raw pixels are all positive, which makes the gradients of a neuron’s weights tend to share one sign and the updates zig-zag; centred inputs avoid that. It also puts the three colour channels on the same scale and matches the input scale that PyTorch’s default initialisation assumes. The effect is faster, more stable training, not a change in what the network can represent.
- **No augmentation:** data augmentation means creating extra, slightly altered training images on the fly (random horizontal flips, random crops, small rotations, colour jitter). The label stays the same, but the network sees a different version each epoch, so it is much harder to memorise. It is a strong regulariser (see below). Removing it makes overfitting easier to see, which is what this assignment wants to study.

## 2.4 Training vocabulary

- **Epoch**: one full pass over the 12 000 training images.
- **Batch (mini-batch)**: the group of images used for one weight update. Here the batch size is 64, so one epoch has  $\lceil 12000/64 \rceil = 188$  iterations (the last batch has 32 images), and 10 epochs give 1 880 weight updates in total.
- **Gradient descent**: to minimise the loss  $L$ , move each weight  $w$  a small step against its gradient,  $w \leftarrow w - \eta \partial L / \partial w$ , where  $\eta$  is the **learning rate** (here  $10^{-3}$ ). Too large a  $\eta$  overshoots, too small learns slowly. Using a mini-batch to estimate the gradient instead of the whole dataset is called **stochastic** gradient descent (SGD).
- **Backpropagation**: the chain rule applied layer by layer from the output back to the input, to compute  $\partial L / \partial w$  for every weight efficiently. It runs through *every* layer, conv layers included. Each layer with weights (Conv, FC) computes (i) the gradient of its own weights and biases, which the optimiser uses to update them, and (ii) the gradient with respect to its input, which is passed back to the previous layer. Layers without weights only route gradients: Flatten reshapes them back, ReLU passes them where the input was positive and blocks them elsewhere, max pool sends the gradient only to the position that held the maximum in each window (the others get zero), and dropout blocks the units it switched off. A conv weight's gradient is a sum over all spatial positions where the filter was applied and over the batch.
- **Regularisation**: any technique that reduces overfitting, usually by limiting how freely the model can fit the training data. Examples: dropout, data augmentation, weight decay, early stopping, a smaller model.
- **Weight initialisation**: the starting values of the weights. PyTorch's default for conv and linear layers is Kaiming-uniform: weights drawn uniformly from  $[-b, b]$  with  $b \propto 1/\sqrt{\text{fan-in}}$  (fan-in = number of inputs to a neuron). This keeps the size of activations roughly stable from layer to layer at the start. Different random draws give slightly different final results, hence the fixed seed.

## 2.5 Cross-entropy loss

The network's last layer produces 10 raw scores (logits). Softmax turns them into probabilities that sum to 1:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}, \quad z_i = \text{logit for class } i.$$

For a single image with true class  $c$ , the cross-entropy loss is

$$L = -\ln p_c.$$

If the network puts probability 0.9 on the right class, the loss is 0.105. If it puts 0.1 (a random guess), the loss is 2.303. If it is confidently wrong, say 0.01, the loss is 4.6.

Two consequences that matter for interpretation:

- Loss can get *worse* while accuracy stays the same or even improves, because the loss penalises overconfident wrong answers heavily. This is the classic sign of overfitting: validation accuracy is flat, but validation loss rises.
- Accuracy only counts right or wrong. Loss also reflects confidence. That is why the assignment asks for both.

## 2.6 Adam

Plain gradient descent uses the same learning rate for every weight. Adam keeps running averages of each weight's gradient  $g_t$  (momentum) and of its square, and scales each weight's step accordingly:

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{(average of gradients)} \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{(average of squared gradients)} \\\hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} && \text{(bias correction, since } m_0 = v_0 = 0) \\w_t &= w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\end{aligned}$$

The PyTorch defaults are  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ , and here  $\eta = 10^{-3}$ . A weight with consistently large gradients has a large  $\sqrt{\hat{v}_t}$ , so its step is shrunk; a weight with small gradients gets a relatively bigger step. In practice this needs little tuning, which is why it is the default choice. For the viva, “adaptive per-weight step sizes plus momentum” is enough.

## 2.7 Dropout

During training, dropout with  $p = 0.5$  randomly sets half of a layer's activations to zero on each forward pass. The network cannot rely on any single neuron, so it learns more redundant, robust features. This reduces overfitting.

At evaluation time (`model.eval()`), dropout is switched off and all neurons are used. This has an important consequence for reading the results:

The “training accuracy” you measure *during* an epoch is computed with dropout on and with weights that are still changing, so it is a pessimistic estimate. The same training images evaluated with dropout off after training will give a higher accuracy.

This is why the results table reports the training accuracy measured in evaluation mode, while the epoch-by-epoch curves use the running values.

# 3 The architecture

## 3.1 Reading the layers

The baseline in the brief is:

Conv(3→32, 5×5) → Pool → Conv(32→64, 5×5) → Pool → Conv(64→128, 3×3) → Conv(128→128, 3×3) → Conv(128→128, 3×3) → Pool → Flatten → FC(8192→512) → FC(512→256) → FC(256→10)

with ReLU after every layer except the last, and dropout(0.5) after the first two FC layers.

Think of it in two halves:

- **Feature extractor** (the five conv layers). It turns pixels into a compact set of learned features: edges and colours early, textures and parts later.
- **Classifier** (the FC layers). It takes those features and decides which of the 10 classes they most resemble.

### 3.2 Why the sizes are 64, 32, 16, 8

For a convolution or pooling layer, the output width is

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - F + 2P}{S} \right\rfloor + 1.$$

For a  $5 \times 5$  kernel with padding 2 and stride 1:  $(64 - 5 + 4)/1 + 1 = 64$ . For a  $3 \times 3$  kernel with padding 1: also unchanged. Such convolutions are called “same” convolutions. So only the max pools shrink the image: each  $2 \times 2$  pool halves the height and width. There are three pools, so

$$64 \rightarrow 32 \rightarrow 16 \rightarrow 8.$$

The last conv layer has 128 channels, so flattening gives  $128 \times 8 \times 8 = 8192$  numbers. In Experiment 3 it has 64 channels, so the flattened size is  $64 \times 8 \times 8 = 4096$ . This is exactly why the first FC layer’s input size changes in Experiment 3, and it is a common source of errors.

### 3.3 Flatten and fully connected layers

**Flatten** reshapes the last conv/pool output, a block of  $\text{channels} \times \text{height} \times \text{width}$  ( $128 \times 8 \times 8$ ), into one vector (8192 numbers). It has no weights and does no arithmetic.

A **fully connected (FC, Linear) layer** connects every input to every output:  $y_j = \sum_i w_{ji}x_i + b_j$ .  $\text{FC}(n \rightarrow m)$  has  $nm + m$  parameters and costs  $nm$  MACs per image. Compared with a conv layer, where a small filter sees only a local patch and is reused at every position, an FC weight is separate for each input–output pair and used once. So FC layers are parameter-heavy but cheap in arithmetic, and conv layers are the opposite. The convs form the *feature extractor*; the FC layers are the *classifier* that combines all features into 10 class scores (logits).

### 3.4 Is the baseline the real AlexNet?

No, it is a small AlexNet-*style* network. It keeps the ideas (stacked convs, max pools, ReLU, dropout, large FC head) but is scaled down:

|                | Real AlexNet (2012)                   | This baseline                       |
|----------------|---------------------------------------|-------------------------------------|
| Input, classes | $224 \times 224$ , 1000               | $64 \times 64$ , 10                 |
| Conv1          | $11 \times 11$ , 96 filters, stride 4 | $5 \times 5$ , 32 filters, stride 1 |
| Conv channels  | 96, 256, 384, 384, 256                | 32, 64, 128, 128, 128               |
| FC layers      | 4096, 4096, 1000                      | 512, 256, 10                        |
| Parameters     | about 61M                             | 4.75M                               |
| Extras         | local response norm., two GPUs        | none                                |

### 3.5 What ReLU does

ReLU is  $\max(0, x)$ . Without a nonlinearity between layers, a stack of linear layers collapses into a single linear layer, no matter how deep. ReLU is cheap and does not saturate for positive inputs, so gradients flow well through many layers.

## 4 Parameters versus computation

This section is the core of the analysis. Learn it well.

## 4.1 Counting parameters

Notation used in these notes (as taught in class):  $K$  = number of filters,  $F$  = filter (kernel) size, so each filter is  $F \times F$ , and  $d$  = depth (number of channels) of the input. A conv layer with  $K$  filters of size  $F \times F$  on an input of depth  $d$  has

$$K(F^2d + 1)$$

parameters: each filter has  $F^2d$  weights plus one bias, and there are  $K$  filters. An FC layer from  $n$  to  $m$  units has  $m(n + 1) = n \cdot m + m$  parameters.

| Layer             | Baseline         | Exp 2            | Exp 3            |
|-------------------|------------------|------------------|------------------|
| Conv1             | 2 432            | 2 432            | 1 216            |
| Conv2             | 51 264           | 51 264           | 12 832           |
| Conv3             | 73 856           | 73 856           | 18 496           |
| Conv4             | 147 584          | 147 584          | 36 928           |
| Conv5             | 147 584          | 147 584          | 36 928           |
| <i>Conv total</i> | 422 720          | 422 720          | 106 400          |
| FC1               | 4 194 816        | 2 097 408        | 2 097 664        |
| FC2               | 131 328          | 32 896           | 131 328          |
| FC3               | 2 570            | 1 290            | 2 570            |
| <i>FC total</i>   | 4 328 714        | 2 131 594        | 2 231 562        |
| <b>Total</b>      | <b>4 751 434</b> | <b>2 554 314</b> | <b>2 337 962</b> |

Observations worth memorising:

- In the baseline, about **91%** of the parameters are in the FC head, and FC1 alone holds 4.19 million of them. The five conv layers together have only 0.42 million.
- Experiment 2 attacks the big block directly, so the parameter count drops by about 46%.
- Experiment 3 shrinks the conv layers to a quarter of their size (halving both input and output channels roughly quarters the weights), but the total only falls by about 51% because FC1 is still huge.

## 4.2 Counting computation

Parameters measure *memory*. Computation is a different quantity. A conv filter is *reused at every spatial position*, so a conv layer performs

$$\text{MACs} = H_{out} \cdot W_{out} \cdot K \cdot F^2 \cdot d$$

multiply-accumulate operations (one MAC is one multiplication plus one addition,  $acc \leftarrow acc + a \cdot b$ ; a MAC is about 2 FLOPs), while an FC weight is used exactly once per image. Early conv layers run on large feature maps ( $64 \times 64 = 4096$  positions), so a small number of weights does a lot of work.

| Layer (output size)  | Baseline       | Exp 2          | Exp 3         |
|----------------------|----------------|----------------|---------------|
| Conv1 (64×64)        | 9.8 M          | 9.8 M          | 4.9 M         |
| Conv2 (32×32)        | 52.4 M         | 52.4 M         | 13.1 M        |
| Conv3 (16×16)        | 18.9 M         | 18.9 M         | 4.7 M         |
| Conv4 (16×16)        | 37.7 M         | 37.7 M         | 9.4 M         |
| Conv5 (16×16)        | 37.7 M         | 37.7 M         | 9.4 M         |
| <i>Conv total</i>    | 156.6 M        | 156.6 M        | 41.6 M        |
| FC total             | 4.3 M          | 2.1 M          | 2.2 M         |
| <b>Total MACs</b>    | <b>161.0 M</b> | <b>158.8 M</b> | <b>43.8 M</b> |
| Relative to baseline | 100%           | 99%            | 27%           |

This is a striking mismatch:

- In the baseline, **91% of the parameters are in the FC head, but 97% of the arithmetic is in the conv layers.**
- Experiment 2 halves the parameters but saves only about 1% of the compute.
- Experiment 3 has slightly *fewer* parameters than Experiment 2, but needs only about a *quarter* of the computation.

So “fewer parameters” does not mean “faster”. That is the central message of the cost analysis. You can quote it as: *parameters cost memory, convolutions cost time.*

Why is Conv2 the most expensive layer? It uses  $5 \times 5$  kernels (25 weights per channel, versus 9) on a  $32 \times 32$  map with 32 input and 64 output channels. The count is  $1024 \times 32 \times 64 \times 25 = 52.4 \text{ M}$ .

### 4.3 Backward pass

Training runs a forward pass and a backward pass. The backward pass costs roughly twice the forward pass, so training time scales roughly like the MAC counts above. Your timing should therefore show Experiment 3 clearly faster than the other two, and Experiment 2 only slightly faster than the baseline, if at all. Small differences between the baseline and Experiment 2 can be swamped by timing noise from the machine, so be careful not to over-interpret them.

## 5 Generalisation, overfitting and convergence

### 5.1 Definitions

- **Underfitting:** the model cannot even fit the training data. Both training and validation accuracy are low, and training loss is still high.
- **Overfitting:** the model fits the training data well but performs noticeably worse on unseen data. Training accuracy keeps rising while validation accuracy plateaus, and validation loss stops falling and begins to *rise*.
- **Generalisation gap:** training accuracy minus validation accuracy. A large positive gap means overfitting.
- **Convergence:** training has settled, meaning curves have flattened and further epochs would change little. If the training loss is still dropping steeply at epoch 10, the model has not converged and 10 epochs was too few to see its final behaviour.

## 5.2 Why overfitting is likely in this setup

Consider the ingredients: only 12 000 training images, no augmentation, a network with 4.75 million parameters (roughly 400 parameters per training image), and Adam at a fairly high learning rate. Such a network has enough capacity to start memorising the training set within a few epochs. Dropout helps but will not fully prevent it.

## 5.3 How to read the curves

| What you see                                  | What it means                                                                                       |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Both losses falling together                  | Still learning general patterns.                                                                    |
| Training loss falling, validation loss flat   | Starting to overfit.                                                                                |
| Training loss falling, validation loss rising | Overfitting is under way. The epoch with the lowest validation loss is where useful learning ended. |
| Both accuracies low and close                 | Underfitting, or the model is too small.                                                            |
| Curves flat near the end                      | Converged.                                                                                          |

# 6 Expected results and the reasoning behind them

*Important: everything in this section is a prediction made before looking at the results, based on general experience with small CNNs on CIFAR-10. It is here so that you can tell whether your actual results are sensible. The report must be written from the actual numbers, and if they disagree with these predictions, explain why rather than forcing them to match.*

## 6.1 Experiment 1: baseline

- **Epoch 1:** accuracy well above 10%, likely 35 to 50%; loss starting near 2.3 and dropping below 1.7.
- **Validation accuracy:** rises quickly over the first few epochs and then flattens. A final value in the region of 65 to 75% is plausible for this setup. This is far below the 90%+ that modern networks reach, because the training set is small and there is no augmentation.
- **Training accuracy:** keeps climbing, potentially reaching 85 to 95% by epoch 10.
- **Validation loss:** likely reaches its minimum around epochs 3 to 6 and then rises.
- **Conclusion I would expect:** clear signs of overfitting in the later epochs, with a generalisation gap of perhaps 15 to 25 points. Training has converged in the sense that the training data is nearly memorised, but validation performance has stopped improving.

## 6.2 Experiment 2: smaller classifier

Reasoning: the first FC layer is a huge matrix (4.19M weights) that connects every one of the 8192 conv features to 512 units. The information the classifier needs is mostly already in the 8192 features. Halving the width to 256 units discards a lot of capacity in a place where it is probably redundant.

- **Parameters:** 2.55 M, about 46% lower.
- **Validation accuracy:** about the same as the baseline, likely within  $\pm 2$  points. Given the sampling noise of roughly 1 point on 2 000 images, a difference in this range is not evidence that one is better.

- **Training accuracy and gap:** possibly slightly lower, because there is less capacity to memorise, and perhaps a slightly smaller gap.
- **Training time:** nearly unchanged, since the conv layers dominate the cost.
- **Interpretation:** most of the baseline’s classifier capacity was not needed. Most of the model’s *useful* capacity sits in the feature extractor, and the FC head is largely spare capacity that mainly helps memorisation.

### 6.3 Experiment 3: narrower convolutions

Reasoning: with half the filters, each layer can detect fewer distinct patterns, so the feature representation is poorer. The classifier receives 4096 features instead of 8192, and those features carry less information.

- **Parameters:** 2.34 M, roughly half of the baseline.
- **Compute and training time:** about a quarter of the MACs, so a substantially shorter training time. In my quick timing tests on this machine a training step for Experiment 3 took about one third of the baseline’s time, which supports this.
- **Validation accuracy:** plausibly a little lower than the baseline, perhaps by 1 to 5 points, because the features are less expressive. It could also be close to equal, since this is a hard task for a small dataset and both networks are limited by the data rather than the capacity.
- **Generalisation gap:** probably smaller, as there is less capacity to memorise. Note that FC1 is still large (2.1M weights), so the model can still overfit substantially.
- **Interpretation:** the conv layers carry the representational power. Cutting them costs some accuracy, but buys a large reduction in computation, which is a much better trade than Experiment 2 offers on the compute axis.

### 6.4 The comparison you are ultimately asked to make

|                        | Baseline  | Exp 2              | Exp 3                  |
|------------------------|-----------|--------------------|------------------------|
| Parameters             | 4.75 M    | 2.55 M             | 2.34 M                 |
| MACs per image         | 161 M     | 159 M              | 44 M                   |
| Expected val. accuracy | reference | ≈ same             | same or slightly lower |
| Expected gap           | largest   | similar or smaller | smaller                |
| Expected time          | reference | ≈ same             | much lower             |

The story to tell: Experiment 2 saves memory but not time; Experiment 3 saves both, at some risk to accuracy. Neither model was tuned, so a model that matches the baseline in accuracy with a smaller footprint is the better design.

### 6.5 What actually happened (local CPU run)

| Model    | Train acc | Val acc | Test acc | Val loss | Gap  | Params | Time (s) |
|----------|-----------|---------|----------|----------|------|--------|----------|
| Baseline | 83.7%     | 63.4%   | 62.5%    | 1.097    | 20.3 | 4.75M  | 445      |
| Exp 2    | 75.8%     | 61.2%   | 62.0%    | 1.133    | 14.6 | 2.55M  | 444      |
| Exp 3    | 80.6%     | 61.3%   | 63.3%    | 1.153    | 19.3 | 2.34M  | 164      |

Compared with the predictions above:

- **Right:** Exp 2 halves the parameters but the training time is unchanged (444 s vs 445 s). Exp 3 is 2.7 times faster. Both reduced models have similar validation accuracy to the baseline within about 2 points, and Exp 2 has the smallest gap.
- **Wrong:** validation accuracy was 61 to 63%, below the 65 to 75% I guessed. Validation loss did *not* bottom out at epochs 3 to 6 and then rise. In the baseline it kept falling to epoch 9 and was flat after that, while the training loss and training accuracy were still improving at epoch 10. The honest description is: clear generalisation gap, early overfitting (validation loss flat while training loss falls), and not fully converged in 10 epochs.
- **Speed-up smaller than the MAC ratio:** Exp 3 has 27% of the MACs but took 37% of the time (164 vs 445 s), since FC layers, data handling and other overheads do not shrink.
- **Noise:** the Exp 3 validation accuracy went from 64.0% at epoch 9 to 61.3% at epoch 10, and the test accuracies (62.5, 62.0, 63.3) reverse the validation ordering. So 2-point differences between models are suggestive, not conclusive. The dataset was a JPEG copy of CIFAR-10.

## 7 Reading the three plots

1. **Accuracy versus epoch** (train and validation for all models). Look for: how fast each model rises, where validation accuracy flattens, and how far the training curve pulls away from the validation curve. The distance is the generalisation gap.
2. **Loss versus epoch.** Look for the epoch where validation loss bottoms out and starts rising. That is your evidence for overfitting. Compare how early it happens in each model.
3. **Validation accuracy versus number of parameters.** Three points. If Experiment 2 and Experiment 3 sit to the left of the baseline at about the same height, then parameters were being wasted. Remember that this plot has *parameters* on the x-axis, not compute. Experiment 3 and Experiment 2 are close together horizontally, but very different in computational cost. Say this explicitly in the analysis, because the plot alone hides it.

## 8 The proposed modification

The assignment wants *one* additional modification, argued but *not trained*. Because you cannot show numbers, the marks come from the quality of the reasoning.

### 8.1 A candidate: add batch normalisation after each convolution

**What it does.** Batch normalisation standardises each channel's outputs over the current mini-batch, then applies a learned scale  $\gamma$  and shift  $\beta$ :

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta,$$

where  $\mu_B, \sigma_B^2$  are the mean and variance of that channel over the batch (at evaluation time, running averages stored during training are used instead). It adds only  $2K$  parameters per conv layer (with  $K$  filters), which is a few hundred numbers in total, and a very small amount of extra computation.

**Why it should help the accuracy–computation trade-off.**

- It stabilises the distribution of each layer’s inputs, which typically allows faster convergence at the same learning rate. In a fixed 10-epoch budget, that means more useful progress per epoch.
- It has a mild regularising effect, because each sample is normalised using noisy batch statistics. That can slightly reduce the generalisation gap.
- Since it improves accuracy for almost no extra cost, it makes the *narrow* network of Experiment 3 more competitive. The reduced-width model already saves about 73% of the computation, so recovering some of its lost accuracy would move it further toward a better trade-off.

**Expected effect.** Faster early convergence, validation accuracy a few points higher than the same model without it, a negligible change in parameters (well under 1%) and a small increase in run time from the normalisation operations.

**Risks to mention.** Batch statistics are noisy with small batches (64 is fine). Normalisation behaves differently in training and evaluation mode, so `model.eval()` matters even more. It does not remove the large FC head’s tendency to overfit.

## 8.2 Alternatives, if you want a second angle

- **Global average pooling instead of Flatten + FC1:** averages each  $8 \times 8$  channel down to one number, so 128 channels give a 128-vector instead of 8192. This removes the 4.19M-weight layer entirely. Very large parameter saving, but accuracy changes in a less predictable way, since spatial layout is thrown away.
- **Depthwise-separable convolutions:** split a conv into a per-channel  $F \times F$  filter (depth-wise) followed by a  $1 \times 1$  conv that mixes channels (pointwise). The cost per position falls from  $KF^2d$  to  $F^2d + Kd$ , about  $1/F^2$  of the original for large  $K$  (roughly 9 times fewer for  $3 \times 3$ ). This targets the real source of computation.
- **Replacing the  $5 \times 5$  Conv2 with two stacked  $3 \times 3$  convs:** the *receptive field* (the input region that influences one output value) is the same,  $5 \times 5$ , but the weights per channel pair drop from 25 to  $2 \times 9 = 18$ , and there is an extra ReLU in between. This is the idea behind VGG.

Pick one and defend it; do not list all of them in the final analysis.

## 9 Likely questions and short answers

**Why did you reset the seed before each model?** So all models start from the same position in the random stream, and differences come from the architecture, not from luck in the initial weights.

**Why is the test set only used at the end?** If it influenced any decision, its accuracy would be an optimistic estimate of real-world performance.

**Why can validation loss rise while accuracy stays flat?** The model becomes overconfident on the examples it gets wrong. Loss penalises confident mistakes heavily, but accuracy does not care about confidence.

**Why is the training accuracy in the table higher than in the plot?** The plot uses running values measured with dropout on while weights were changing. The table uses a separate pass in evaluation mode after training.

**Does fewer parameters mean faster training?** No. Convolutions reuse each weight across many positions, so they dominate the computation even though they hold few parameters. Experiment 2 and Experiment 3 have similar parameter counts but very different costs.

**Is a 1-point difference between models meaningful?** Not here. Validation accuracy on 2000 images has a sampling error of about 1 point, and there is only one run per model.

**What limits the accuracy overall?** Small training set (12 000), no augmentation, only 10 epochs, and upsampled  $32 \times 32$  images that contain no extra detail.

**Is the backward pass only in the FC layers?** No. Backprop goes through every layer. Conv and FC layers update their weights and biases; max pool, ReLU, Flatten and dropout have no weights and only route the gradient (max pool sends it to the position that won the max).

**What does max pool update?** Nothing; it has no parameters. In the backward pass the gradient goes only to the maximum position in each window.

**What is a MAC?** One multiply plus one add,  $acc \leftarrow acc + a \cdot b$ . A conv layer costs  $H_{out}W_{out}KF^2d$  MACs;  $FC(n \rightarrow m)$  costs  $nm$ .

**Is the baseline AlexNet?** No, it is a small AlexNet-style network (4.75M parameters,  $64 \times 64$  input, 10 classes) versus the original 61M-parameter ImageNet model.

**How does normalisation help?** Input normalisation centres and scales pixels so gradients are balanced and training is faster and more stable. Batch norm does the same inside the network for each layer's inputs, allowing faster convergence and adding mild regularisation.

**Why does Experiment 3 change the size of FC1's input?** Halving the last conv layer's filters from 128 to 64 reduces the flattened feature vector from  $128 \cdot 64 = 8192$  to  $64 \cdot 64 = 4096$ .